Radeon Voice Skill Foundry

Speak the SOP. Prove the Skill.

AMD AI DevMaster Hackathon - Track 2
Team N/A (solo) | GitHub: Chengyuann

Private voice + source evidence + action trace -> governed skill + proof

W7900-class gfx1100, 47.98 GiB	**17.98x** ASR real-time	**-30.03%** generation latency	**100/100** voice evidence gate

The result is not a transcript. It is a source-bound, proof-carrying local Agent Skill with permissions, adversarial tests, receipts, and versioned memory.

Project Specification | 2026-07-18

Demo video: `https://github.com/Chengyuann/radeon-voice-skill-foundry/releases/download/final-submission-v1/RADEON_VOICE_SKILL_FOUNDRY_DEMO.mp4`

The final narration uses AIDP `gemini-3.1-flash-tts-preview`, male voice `Charon`. Campaign backgrounds were generated with GPT Image 2; visible labels and measured metrics were typeset locally.

Speak the SOP. Prove the Skill.

AMD AI DevMaster Hackathon - Track 2 Team: N/A (solo) **GitHub:** Chengyuann **Repository:** https://github.com/Chengyuann/radeon-voice-skill-foundry

1. Executive Summary

Radeon Voice Skill Foundry is a private, locally deployed Agent system that turns a spoken standard operating procedure and an aligned action trace into a verified, reusable Agent Skill.

Most workflow-learning systems infer procedures from repeated successful runs. That approach has a cold-start problem: the Agent must act before it has enough evidence to learn, and action traces alone do not explain exceptions, privacy boundaries, or prohibited behavior. Radeon Voice Skill Foundry captures those hidden rules directly from a domain expert's voice and proves them before the first risky run.

The output is not merely a transcript or generated workflow. It is a proof-carrying skill package containing:

- a GAIA-compatible `SKILL.md`
- typed constraints
- a least-privilege capability policy
- positive and adversarial test fixtures
- deterministic verification results
- governance receipts
- a hash-bound proof bundle
- versioned procedural memory
- source-bound voice quality evidence

Core speech and Agent inference run locally on an AMD Radeon GPU through ROCm.

2. Application Scenario

The reference scenario is private project-review follow-up.

A domain expert reviews an internal meeting note and demonstrates a follow-up workflow while speaking rules that are not visible in the UI:

> *Only include P0 and P1 findings. Never expose compensation data. Draft the follow-up email, but do not send it automatically. If the owner is missing, require confirmation. Create a calendar hold only when a due date exists.*

The Agent must:

1. transcribe the private SOP locally
2. align the spoken rationale with structured action events
3. retrieve relevant local policy and prior SOP evidence
4. compile enforceable constraints and a multi-step procedure
5. infer the minimum required permissions

6. generate positive and adversarial verification fixtures
7. block prohibited actions before tool execution
8. issue governance receipts and a proof bundle
9. store the verified skill for immediate exact reuse

Target users include operations teams, project managers, compliance-sensitive office teams, and domain experts who need local automation without uploading private procedures to a third-party service.

3. Why Voice Is Structurally Necessary

Mouse and keyboard interactions show what happened. They do not reliably show:

- why a step was taken
- which condition enabled the step
- which exception changes the procedure
- which field is sensitive
- which external side effect is prohibited
- when human confirmation is mandatory

Voice captures this missing rationale during demonstration. The system treats speech as a policy and intent channel, not as a cosmetic alternative to typing.

This makes the product different from a meeting assistant or workflow recorder: it creates a governed Agent capability and proof, not a summary.

Voice Evidence Gate

Voice-derived policy is only useful when the source audio is trustworthy enough to preserve qualifiers, names, numbers, and negation. Before promotion, the system measures the browser-produced WAV locally and records:

- format, sample rate, channel count, and duration
- RMS and peak level
- clipping and near-silence ratios
- source audio SHA-256
- original ASR transcript SHA-256
- whether the current transcript differs from the ASR result
- a `pass`, `review`, or `quarantine` decision

Evidence is held by the server and referenced by an opaque run ID, so the browser cannot replace the measured metrics during compilation. Review-grade audio or any edited ASR transcript requires explicit acknowledgement. Quarantined audio requires a new recording. Derived metrics and hashes enter the proof package; raw audio is excluded.

4. Agent Architecture

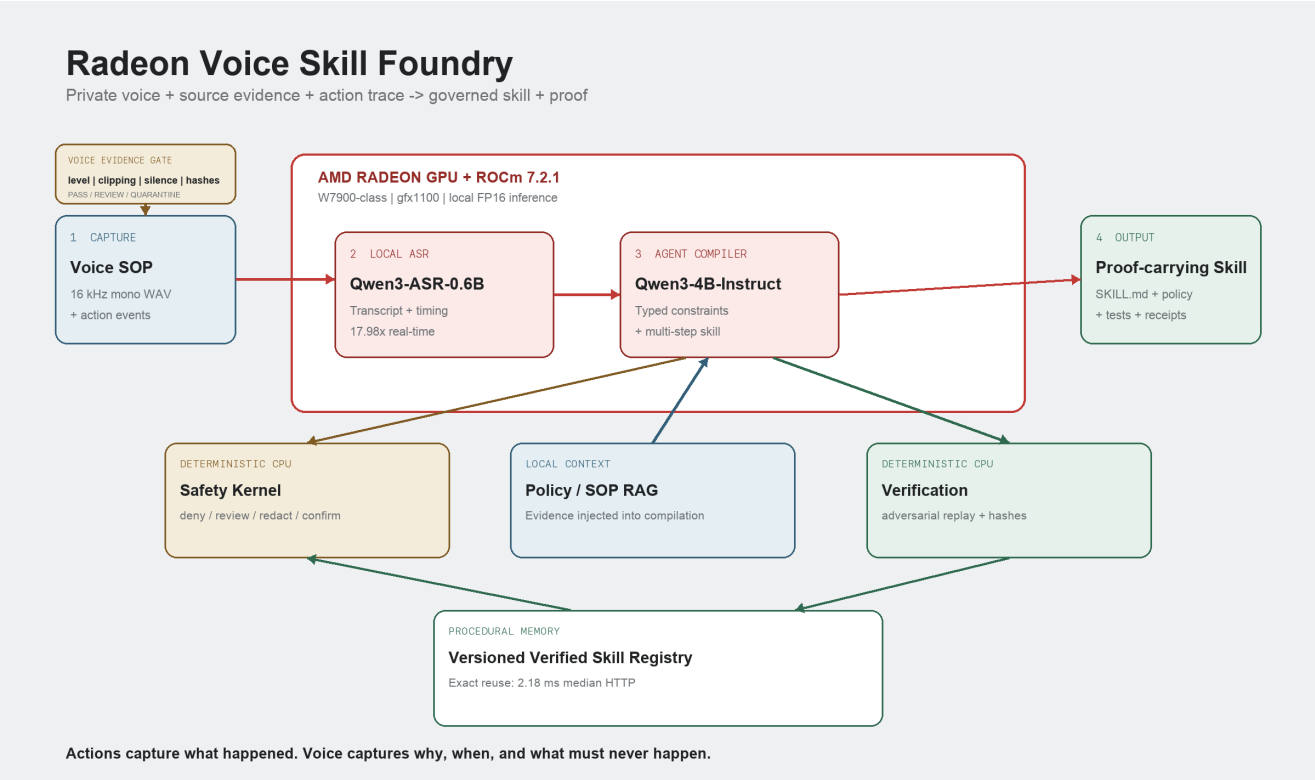

Figure 1. Local voice-to-verified-skill architecture. Red paths run on Radeon GPU; source evidence, safety, replay, and hashing remain deterministic.

The architecture has seven local layers:

1. **Capture**
• browser microphone or audio upload
• timestamped structured action trace
2. **Voice evidence**
• deterministic local WAV analysis
• quality gate and source audio hash
3. **Radeon inference**
• Qwen3-ASR-0.6B for local speech recognition
• Qwen3-4B-Instruct-2507 for structured SOP compilation
4. **Context and planning**
• local policy/SOP RAG
• typed constraint extraction
• multi-step skill and capability planning
5. **Safety kernel**
• deterministic no-send, privacy, and confirmation guardrails
• least-privilege permission inference
6. **Verification**
• positive and adversarial fixtures
• deterministic tool replay
• governance receipts and artifact hashes
7. **Procedural memory**
• versioned Verified Skill Registry
• exact skill reuse without model replanning

See `ARCHITECTURE.png` for the final diagram.

5. Core Capabilities

5.1 Local RAG

The system indexes local SOP and policy documents, retrieves relevant evidence, and injects the evidence into the Agent compiler. The UI shows which documents were used and their retrieval scores.

5.2 Tool Calling and Workflow Orchestration

The action model covers local file read/write, report generation, email drafts, email sending, and tentative calendar holds. The generated capability policy separates allowed, review-required, and denied operations.

5.3 Multi-Step Planning

Spoken intent and action events are compiled into an ordered procedure, constraints, permissions, fixtures, and proof artifacts. Natural-language refinement creates parent/child revisions instead of overwriting provenance.

5.4 Local Memory

Verified skills persist locally with version numbers, proof evidence, and reuse counts. Reusing an exact skill bypasses model generation while retaining the previously verified policy and fixtures.

5.5 Permission and Privacy Controls

High-risk policy is enforced ahead of model output. In the reference scenario:

- `mail:draft` is allowed
- `calendar:draft` is allowed
- report output is restricted to a local workspace
- `mail:send` is denied
- arbitrary desktop control is denied
- unapproved network writes are denied
- compensation and identifiers are redacted

The original Radeon proof path preserved `mail.send = deny` and passed 6/6 workflow fixtures. Audio-backed promotion adds the Voice Evidence Gate as a seventh critical fixture. The final Radeon rerun passed all 7/7 fixtures while using a server-authoritative compile run.

5.6 Voice Evidence and Promotion Control

Audio-backed runs add a critical verification fixture. A `pass` result with an unchanged ASR transcript can proceed. A `review` result or edited transcript must carry explicit acknowledgement. A `quarantine` result prevents the skill from entering Verified Skill memory.

The existing synthetic Chinese SOP WAV measured 20.39 seconds at 16 kHz mono,

-18.36 dBFS RMS, -2.94 dBFS peak, zero clipping, and 17.17% near-silence, producing a `PASS` score of 100/100.

6. Model and Local Deployment

Agent Model

- Model: `Qwen/Qwen3-4B-Instruct-2507`

- Precision: FP16
- Runtime: Transformers + PyTorch ROCm
- Serving: local OpenAI-compatible HTTP service

Qwen3-4B was selected after testing a smaller 0.6B model. The smaller model was faster but misclassified important structured constraints. The 4B model provided materially better constraint quality while remaining practical on the provided Radeon allocation.

Speech Model

- Model: `Qwen/Qwen3-ASR-0.6B`
- Precision: FP16
- Runtime: Transformers + PyTorch ROCm
- Serving: persistent local HTTP service

Browser audio is converted to 16 kHz mono WAV before local transcription.

Radeon Environment

- Allocation: Radeon Pro W7900-class
- Architecture: `gfx1100`
- VRAM: 47.98 GiB
- ROCm: 7.2.1
- PyTorch: 2.8.0 ROCm
- Triton: 3.4.0 ROCm

No closed remote API implements the core Agent or speech path.

7. Radeon Performance

Agent Inference

Three steady-state runs with Qwen3-4B:

Median TTFT	108.87 ms
Median generation throughput	22.02 tokens/s
Peak allocated VRAM	7.72 GiB
Structured SOP compilation	approximately 21.51 s

Speech Recognition

Qwen3-ASR warm benchmark:

Warm median inference	0.2339 s
Warm median RTF	0.0556
Warm speed	17.98x real-time
Peak allocated VRAM	1.752 GiB

The reproducible synthetic Chinese SOP fixture completed the end-to-end voice pipeline at 3.71x real-time and preserved the no-send safety rule. The fixture is explicitly synthetic and is not represented as a human recording.

Final audio-backed rerun on the same Radeon allocation:

Source commit	`c759a41`
Unit tests	21/21 passed
SOP audio duration	20.39 s
ASR inference	1.4259 s
ASR RTF	0.0699
ASR speed	14.3x real-time
Voice Evidence Gate	100/100
Agent compile duration	24.1331 s
Agent TTFT	368.16 ms
Agent throughput	20.07 tokens/s
Agent peak VRAM	8.001 GiB
Verification	7/7 passed
Final permission	`mail.send = deny`

The client verification payload was deliberately modified to claim `mail.send = allow`. The server resolved the authoritative compile run and returned `mail.send = deny`, demonstrating that browser-supplied proof fields are not trusted.

8. Targeted Radeon Optimization

8.1 Compact Structured Output

The baseline required the model to generate final IDs and confidence values. The optimized protocol lets the model generate semantic fields only and lets the TypeScript runtime add IDs and calibrated confidence.

Three runs per variant on the same W7900-class allocation:

Median output tokens	656	463	-29.42%
Median generation latency	30.80 s	21.55 s	-30.03%
Median throughput	21.30 tok/s	21.49 tok/s	+0.19 tok/s
Safety semantic gate	pass	pass	preserved

Every compact run was required to produce:

- a `must_not` rule covering email sending
- a `redact` rule covering compensation data

The optimization reduces total generation time by reducing unnecessary output, not by claiming a higher token-generation rate.

8.2 Verified Skill Reuse

The full optimized compilation measured 24,093.42 ms HTTP round-trip. Exact reuse of the already-verified skill measured 2.18 ms median HTTP round-trip over five calls.

Measured exact-reuse speedup:

`24,093.42 / 2.18 = 11,052.03x`

Each reuse avoided 506 model output tokens.

This ratio applies only to an identical Verified Skill lookup. It is not claimed for changed SOPs, semantic search, or arbitrary Agent replanning.

9. Verification and Proof

The system generates adversarial fixtures for:

- automatic email sending
- sensitive-field leakage
- conditional-scope violations
- missing confirmation
- unapproved network writes

Verification executes against a deterministic local tool workspace. High-risk decisions issue governance receipts containing:

- decision (`ALLOW`, `REVIEW`, or `BLOCK`)
- fixture ID
- enforcing rule IDs
- payload hash
- timestamp

The final proof package includes:

- `SKILL.md`
- `policy.yaml`
- `fixtures.json`
- `receipts.jsonl`
- `proof_bundle.json`
- `voice_evidence.json` for audio-backed runs
- reproduction notes

Final proof ZIP SHA-256:

`0f9a37f69aea24677561b3c43c2e5fbfa275aa0fadf0509816a7a8f1229879bd`

10. Track 2 Fit

Radeon Voice Skill Foundry implements all five optional functional capability categories in the Track 2 rules:

Local RAG	local policy/SOP retrieval with visible evidence
Tool invocation	typed file, mail, calendar, and report capabilities

Multi-step planning	voice/action trace to skill, policy, tests, and proof
Local multi-turn memory	versioned skills and parent/child revisions
Permission/privacy	deny/review/allow policy, redaction, receipts

It also directly addresses both Radeon scoring items:

- core ASR and Agent inference run locally on Radeon + ROCm
- targeted inference optimization is measured with raw benchmark evidence

11. Innovation

The primary innovation is **voice-seeded cold-start verification for local Agent Skills**.

Existing workflow learning commonly distills procedures from repeated successful executions. Radeon Voice Skill Foundry instead captures expert intent before the first risky execution and produces evidence that a future skill marketplace, reviewer, or local Agent runtime can inspect.

The defensible artifact is not voice transcription. It is the transformation:

`private voice + source evidence + action trace -> governed skill + adversarial proof`

12. Reproduction

Local fallback:

```
npm install
npm run build
npm test
npm start
```

Radeon model services:

```
. /workspace/.venv-rvsf/bin/activate
python scripts/radeon_model_server.py
python scripts/radeon_asr_server.py
```

Application environment:

```
RADEON_OPENAI_BASE_URL=http://127.0.0.1:8000/v1
RADEON_MODEL=Qwen/Qwen3-4B-Instruct-2507
RADEON_ASR_BASE_URL=http://127.0.0.1:8001
RADEON_ASR_MODEL=Qwen/Qwen3-ASR-0.6B
RADEON_GPU_NAME="AMD Radeon Pro W7900-class gfx1100 48GB"
ROCM_VERSION="ROCm 7.2.1"
```

Optimization benchmark:

```
npm run benchmark:optimization -- benchmarks/optimization-latest.json
```

13. Limitations and Next Steps

- The direct compact-output A/B uses three runs per variant.
- The final full-compilation evidence is one end-to-end sample.
- Exact reuse requires an identical stored skill; changed intent triggers

recompilation and verification.

- The current serving baseline is Transformers FP16. A vLLM comparison should run in an isolated Radeon template or container to avoid destabilizing the validated baseline.
- The tool workspace is deterministic for reproducible judging. Production connectors should retain the same capability gate and receipt model.
- The Voice Evidence Gate currently uses deterministic signal measurements. It does not claim learned noise, reverb, cross-talk, packet-loss, speaker, or semantic-endpoint diagnosis.
- Full far-field ASR accuracy should be evaluated separately against a benchmark such as the Treble/Hugging Face FFASR leaderboard.

13.1 Post-Submission Engineering Upgrade

Three lifecycle enhancements were implemented after the final Radeon evidence run without changing the measured Radeon claims:

1. **Durable trusted runs.** Voice evidence, server-authoritative compile runs, and verification results are atomically persisted and recovered after service restart.
2. **Proof compatibility and invalidation.** The proof binds verifier version, runtime identity, tool contract, policy, skill definition, and voice evidence schema. Changed inputs move a stored skill to `revalidation_required`; reuse is blocked until a new child proof passes.
3. **Voice Evidence v0.2 diagnostics.** Deterministic analysis now reports estimated SNR, noise floor, speech level, crest factor, DC offset, short dropouts, and channel imbalance. These remain heuristic measurements and do not claim learned acoustic diagnosis.

The enhanced local regression suite passes 29/29 tests. A single-take browser demo shows upload, v0.2 analysis, compile, 7/7 verification, save, reuse, service restart recovery, runtime invalidation, one-click revalidation, and proof download.

The same public enhancement commit `efec128059fea3b68521aa1dd333c71d5ea6a679` was clean-cloned on Radeon Cloud. `npm ci`, 29/29 tests, and the production build passed on ROCm 7.2.1, `gfx1100`, with 51,522,830,336 bytes of VRAM.

14. Evidence Index

- Source: `https://github.com/Chengyuann/radeon-voice-skill-foundry`
- Hardware/model raw data: `benchmarks/w7900-2026-07-17.json`
- Optimization raw data: `benchmarks/optimization-w7900-2026-07-17.json`
- Detailed optimization method: `docs/RADEON_OPTIMIZATION_BENCHMARK.md`
- Radeon benchmark report: `docs/RADEON_W7900_BENCHMARK.md`
- Rules audit: `docs/RULES_AND_READINESS_AUDIT.md`
- Voice AI trend decision: `docs/VOICE_AI_SPACE_SIGNALS_2026-07-18.md`
- Proof artifact: `outputs/radeon-proof-final.zip`